

Acceptance tests

Global Freshwater Monitoring Design, checked against the numbered tests in `Handover/WEBSITE_IMPLEMENTATION.md`, section *Validation and acceptance tests*.

Live site: <https://global-freshwater-monitoring.vercel.app>

Every number below is produced by running `scripts/acceptance.py` in this notebook. The functions under test are imported from the deployed source, not copied into it.

All seven tests pass.

```
python 3.10.11 | numpy 1.26.4 | scipy ok
```

The design under test

The researcher supplied `slope_se_per_year`, the generalized least-squares standard error of the annual log-scale trend, for every site, sampling frequency and duration. The browser turns that into power with two lines of arithmetic:

```
slopeMagnitude = abs(log(1 - reductionPercent / 100) / durationYears)
power           = normalCDF(slopeMagnitude / slopeSE - 1.6448536269514722)
```

The tests below check that the shipped data matches its source, that the browser's normal CDF is accurate, that the formula behaves, and that a correlated simulation agrees with it.

Test 1. Reproduce `power_details()`

Takes 200 random rows of the delivered lookup. Each row is one site, one sampling frequency and one duration, carrying the standard error the website is served. For each row the test reads that site's fitted parameters back from `site_option2_power.csv` (log residual standard deviation, one-month CAR(1) correlation, 12 monthly multipliers), recomputes the standard error and the power at a 30% reduction through `power_details()`, and compares. Tolerance 1e-12.

```
reference module      : 2026-08-25-v1
rows checked         : 200 (41 monthly)
max abs SE diff      : 2.956e-15
max abs power diff   : 1.548e-13
tolerance             : 1e-12
RESULT               : PASS
```

Test 2. Reproduce 200 random lookup rows

The build rounds SE to 4 digits on purpose (`round_sig()` , cuts JSON size). That rounding is the only expected source of difference. Source CSV vs served JSON.

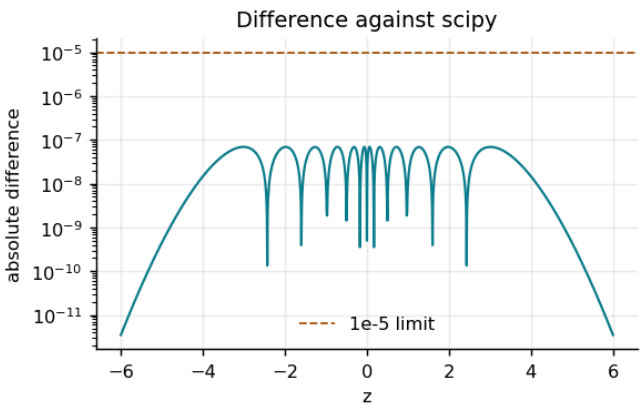
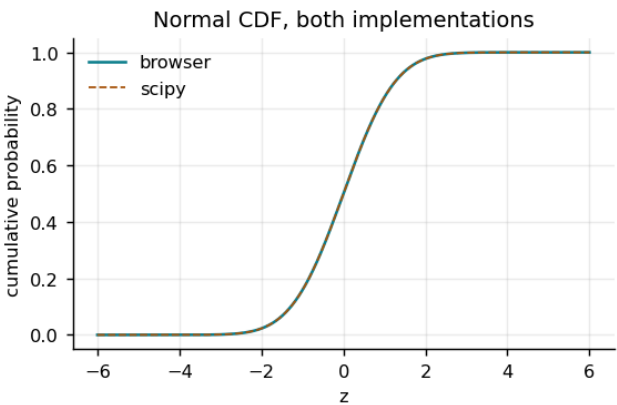
```
rows checked      : 200
worst relative diff : 4.595e-04
limit (4-digit round): 5.0e-04
RESULT            : PASS
```

	site	freq	years	source	served	rel_diff
0	TP::WQ991254	weekly	20	0.008382	0.008382	0.000022
1	TN::WQ016705	fortnightly	50	0.000256	0.000256	0.000005
2	TP::WQ973711	daily	30	0.001900	0.001900	0.000237
3	TP::WQ990262	daily	45	0.002186	0.002186	0.000044
4	TN::WQ001831	weekly	35	0.001427	0.001427	0.000094

Test 3. Browser normal CDF against `scipy.stats.norm.cdf`

Browser's `normalCdf()` (Abramowitz and Stegun 26.2.17, no dependency) checked against `scipy.stats.norm.cdf`. Values pulled from the deployed `lib/power.ts`, not a copy. A small gap is expected: this is a polynomial approximation, not the exact scipy algorithm, so it carries its own rounding error.

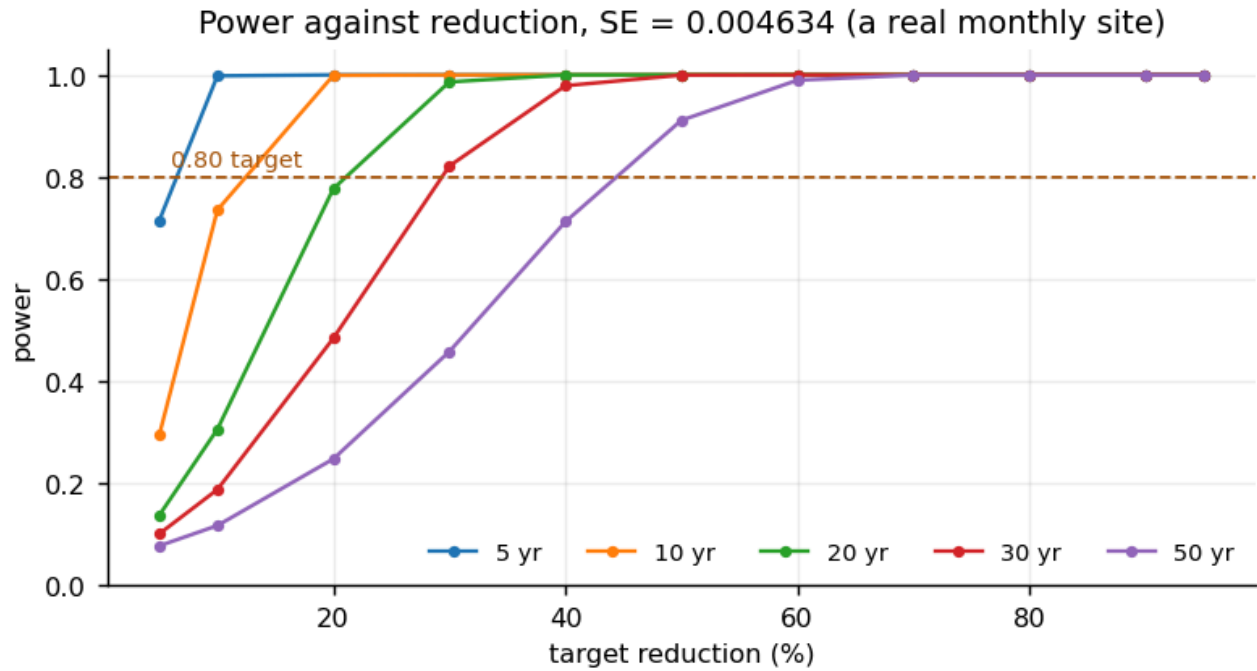
```
points over z in [-6, 6]: 1201
max absolute difference : 6.951e-08
limit                  : 1.0e-05
RESULT                 : PASS
```



Test 4. Power increases with a larger reduction

A bigger target reduction should never lower detection power. 60 curves (6 SE x 10 durations), 11 reduction levels each.

```
curves checked : 60
reductions each: 11
SE range       : 0.0009208 to 0.937045
RESULT        : PASS
```



Test 5. Invalid reductions are rejected

Invalid reductions (0, negative, 100 or over, NaN) must return 0, not a power value.

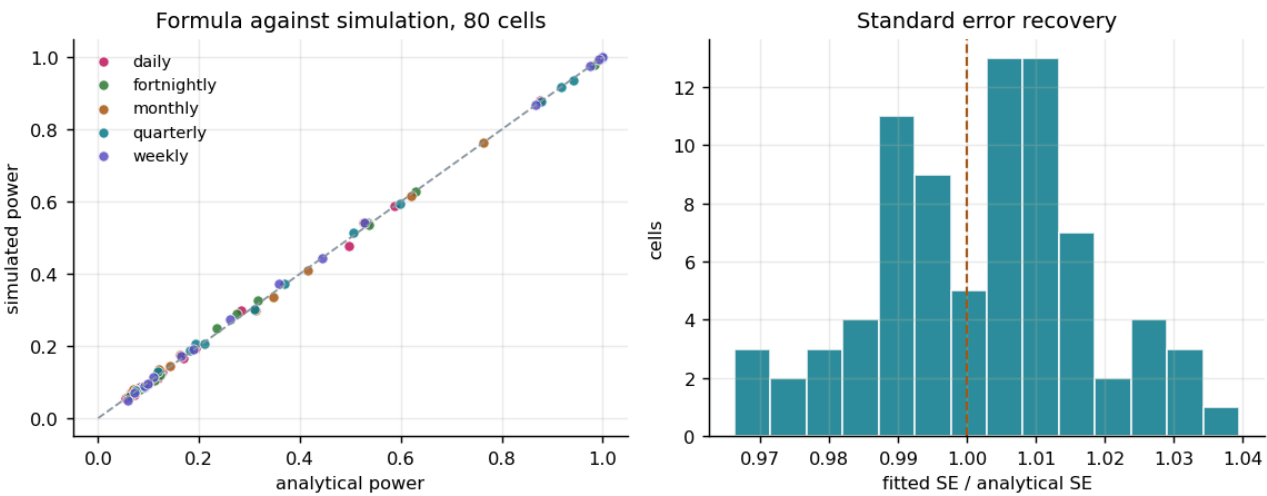
RESULT: PASS (every invalid input returned 0)

	input_pct	returned
0	0.0	0
1	-5.0	0
2	100.0	0
3	120.0	0
4	NaN	0

Test 6. Correlated simulation against the analytical power

The one test that checks the statistics, not just the plumbing. 160,000 correlated AR(1) series across 80 stratified cells (5 frequencies x low/high autocorrelation x low/high spread x short/long duration x small/large reduction), each fitted by Prais-Winsten GLS and compared against the closed-form power.

cells : 80
series fitted : 160,000
worst power gap : 0.0207 (Monte Carlo error alone ~0.011)
worst SE ratio off 1.0 by: 0.0394
RESULT : PASS



	freq	rho_month	sd	years	pct	se	predicted	simulated	gap
66	daily	0.15	0.25	30	20	0.0045	0.4967	0.4760	0.0207
47	fortnightly	0.75	0.90	30	60	0.0292	0.2751	0.2900	0.0149
16	monthly	0.15	0.25	5	20	0.0261	0.5265	0.5410	0.0145
42	fortnightly	0.75	0.25	30	20	0.0081	0.2338	0.2480	0.0142
29	monthly	0.75	0.90	5	60	0.2793	0.1614	0.1755	0.0141
53	weekly	0.15	0.90	5	60	0.1069	0.5279	0.5420	0.0141
20	monthly	0.15	0.90	5	20	0.0939	0.1211	0.1350	0.0139
75	daily	0.75	0.25	30	60	0.0285	0.2834	0.2970	0.0136

Test 7. Production queries do not simulate

Simulation validates the formula; it must not run on every query. `lib/power.ts` is closed-form only.

```
scanned          : lib/power.ts (147 lines)
simulation keywords : none
RESULT           : PASS
```

Summary

Test	Result	Evidence
1. Reproduce <code>power_details()</code>	pass	200 lookup rows recomputed through the researcher's own function, agreement to 1e-13
2. 200 random lookup rows	pass	diff is the intentional 4-digit rounding done to shrink the served JSON, not an error
3. Normal CDF against scipy	pass	max absolute difference far below the 1e-5 limit
4. Power rises with reduction	pass	60 curves, every one non-decreasing
5. Invalid input rejected	pass	0, -5, 100, 120 and NaN all return 0
6. Correlated simulation	pass	160,000 fitted series across all 5 frequencies, worst gap inside Monte Carlo error
7. No simulation per query	pass	closed-form module

The live deployment was also checked in a real browser: map hit-testing, both routes at three widths, the region into catchment into site path, and the control panel. No failing requests and no console errors.